

FlowSync: Intelligent Real-Time Streaming Analytics

An End-to-End Data Engineering & AI Platform for Streaming Events

TEAM MEMBERS

Y. Nihitha (2420090099)

S. Harini (2420030523)

A. Sri Anitha (2420090117)

PROJECT GUIDANCE

Dr. N Shirisha

Associate Professor

Fundamentals of Data Engineering Project Review-3

Objectives

01 Methodology

02 System Design

03 Architecture

04 Use of Modern Tools and Technologies

05 Implementation

06 Code Quality and Fuctionality

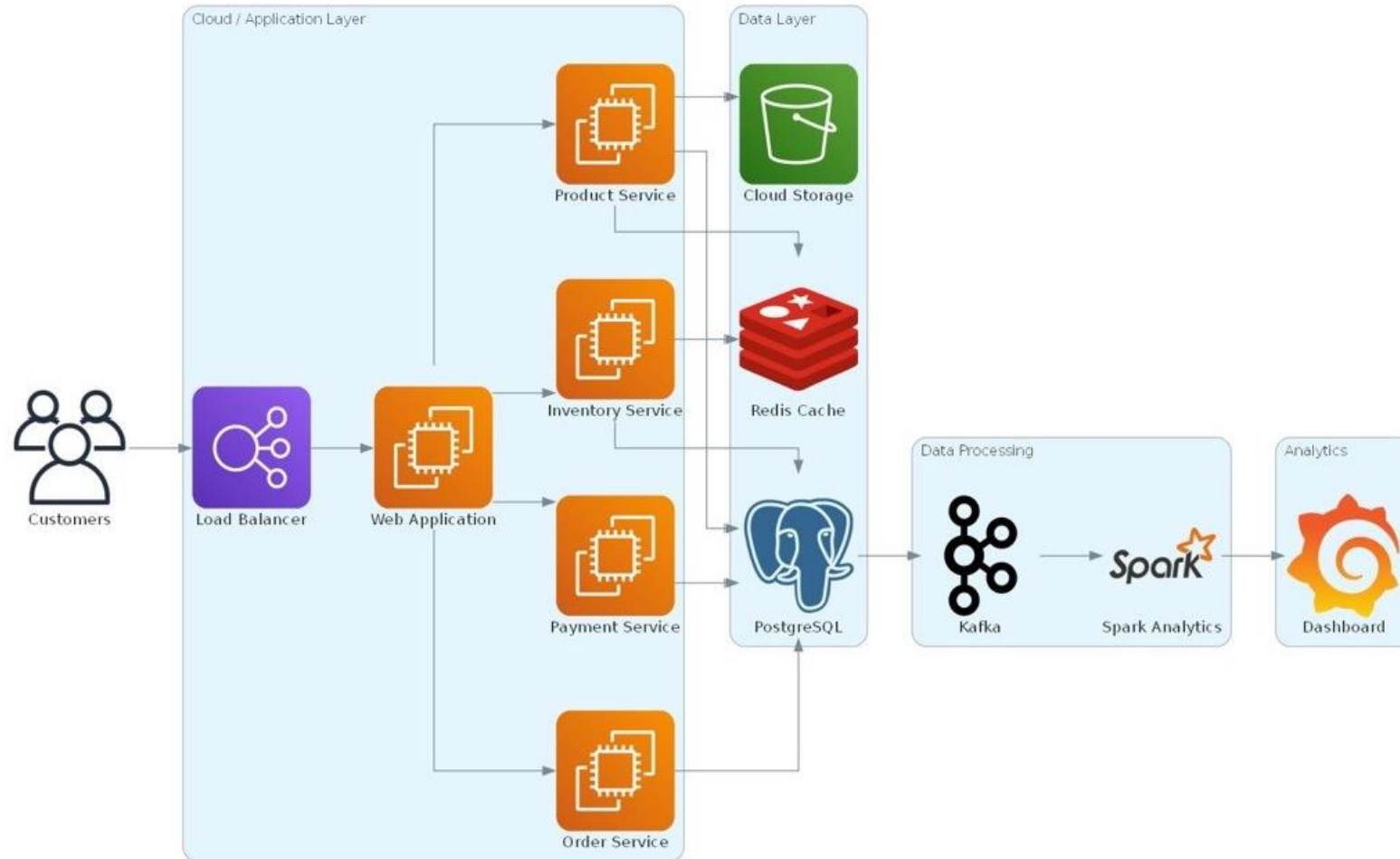
FlowSync follows a structured real-time data engineering methodology designed to continuously collect, process, validate, store, and analyze streaming data. Data from multiple sources such as IoT devices, applications, APIs, and transactional systems is first ingested through Apache Kafka. The incoming streams are then processed using Spark Structured Streaming and PySpark, where data is cleaned, transformed, enriched, and prepared for downstream analysis.

To ensure reliability and consistency, Great Expectations is used for automated data-quality validation, including schema and consistency checks. The validated data is stored using MinIO, Apache Parquet, and PostgreSQL according to the required storage and analytical needs. Scikit-learn is incorporated for anomaly detection, while FastAPI provides API access to processed data and machine-learning results. Finally, Grafana and the FlowSync dashboard provide real-time visualization and monitoring of data quality, system performance, processing activity, and detected anomalies.

Overall methodology:

Data Sources → Kafka Ingestion → Spark Processing → Data Quality Validation → Storage → Anomaly Detection → API → Visualization & Monitoring.

System Design



E-Commerce Website - Data Architecture

- **Data Sources** : IoT, APIs & applications
- **Kafka** : Real-time data ingestion
- **Spark / PySpark** : Stream processing & transformation
- **Great Expectations** : Data quality validation
- **MinIO / Parquet / PostgreSQL** : Data storage
- **Scikit-learn** : Anomaly detection
- **FastAPI** : Backend API
- **Dashboard / Grafana** : Visualization & monitoring
- **Overall Flow:**
Data Sources → Kafka → Spark → Data Quality → Storage & Anomaly Detection → FastAPI → FlowSync Dashboard / Grafana

Technology Stack



Category	Technology	Purpose
Programming	Python, PySpark	Pipeline development and distributed data processing
Data Ingestion	Apache Kafka	Real-time ingestion and event streaming
Stream Processing	Spark Structured Streaming	Low-latency processing and transformation of streaming data
Data Quality	Great Expectations	Data validation, profiling, and quality checks
Data Lake	MinIO (S3-Compatible)	Scalable object storage for raw and processed data
Data Storage	Apache Parquet, PostgreSQL	Efficient analytical storage and structured data persistence
Machine Learning	Scikit-learn	Anomaly detection and machine learning-based analysis
API Layer	FastAPI	High-performance APIs for data and ML services
Monitoring	Grafana	Pipeline, system, and real-time performance monitoring
DevOps & Version Control	Docker, Git	Containerization, deployment, and source-code management

Use of Modern Tools & Technologies

- **Real-time Processing:** Uses Kafka and Spark Structured Streaming for continuous, low-latency data processing.
- **Automated Data Quality:** Great Expectations enables systematic validation and quality checks.
- **Scalable Data Management:** MinIO, Parquet, and PostgreSQL support efficient storage of raw and processed data.
- **Machine Learning Integration:** Scikit-learn is used for detecting anomalies in processed data.
- **API-Based Architecture:** FastAPI enables communication between backend processing and the dashboard.
- **Containerized Development:** Docker supports consistent deployment and environment management.
- **Monitoring & Visualization:** Grafana provides real-time monitoring of pipeline and system performance.

Implementation

- Developed the FlowSync real-time data pipeline using Python and PySpark.
- Implemented Apache Kafka for continuous ingestion and streaming of incoming data.
- Used Spark Structured Streaming to clean, transform, enrich, and process streaming data.
- Integrated Great Expectations to perform automated data validation and quality checks.
- Implemented Scikit-learn for identifying anomalies in processed data.
- Used MinIO, Apache Parquet, and PostgreSQL for storing raw, processed, and structured data.
- Developed FastAPI services to provide processed data and analytical results to the frontend.
- Built a React dashboard prototype to display live events, data quality, anomalies, and system metrics.
- Used Docker and Git to support containerized deployment and version-controlled development.

Overview

Search FlowSync...



Pipeline Connected

SH

Overview

Real-time streaming analytics overview

Prototype Mode



+2.4%

1,285,353

Events Processed

Total streaming events processed



+6.1%

62

Events / Sec

Current simulated throughput



- 0.0%

94.2%

Data Quality

Validated streaming records



-1.2%

1,284

Anomalies Detected

Potential abnormal events

Real-Time Streaming

Simulated ingestion throughput over time

Events Throughput

Events processed per second

LIVE SIMULATION



Overview

Search FlowSync...



Pipeline Connected

SH

Data Quality Overview

Validation outcomes across the current window

Validation Breakdown

Share of records by outcome



Valid

94.2%

Invalid

3.8%

Anomaly Detection

Detected anomalies over the last hour

Anomalies Over Time

Count per 5-minute window



Overview

Q Search FlowSync...



● Pipeline Connected

SH

Anomalies Over Time

Count per 5-minute window



Recent Streaming Events

Newest simulated events across all sources

Event ID	Source	Type	Value	Timestamp	Status
EVT-10517	IoT Sensor	Temperature	28.4°C	09:03:58	Normal
EVT-10516	App Log	Logout	Success	09:03:55	Normal
EVT-10515	IoT Sensor	Pressure	1036.6 hPa	09:03:52	Normal
EVT-10514	App Log	Error	503 Timeout	09:03:49	Normal
EVT-10513	IoT Sensor	Pressure	986.4 hPa	09:03:46	Normal

Code Quality and Functionality



Code Quality

- Modular Design: Pipeline components are separated into independent modules for easier maintenance.
- Readable & Structured Code: Clear naming, organized functions, and consistent coding practices are followed.
- Error Handling: Validation and exception handling help manage invalid or unexpected data.
- Code Quality Tools: ESLint/Oxlint can be used to maintain consistent frontend code quality.
- Version Control: Git is used for tracking changes and managing development.

Functionality

- Real-Time Data Ingestion: Kafka continuously receives streaming data.
- Stream Processing: Spark processes and transforms incoming data in real time.
- Data Validation: Great Expectations checks data quality and consistency.
- Anomaly Detection: Scikit-learn identifies unusual data patterns.
- API Integration: FastAPI exposes processed data and results.
- Dashboard: React prototype displays events, quality metrics, anomalies, and system status.
- Monitoring: Grafana supports pipeline and system-performance monitoring.

Thank You!

FlowSync: Intelligent Real-Time Streaming Analytics

Course Code: 24DEA3101 | Guide: Dr. N Shirisha